A Production-Ready Machine Learning-Based Bank Fraud Detection System with Real-Time Analytics and Enterprise Security

Vibha Sabareesan
Dept. of Computer Science and Engineering
Jain University, Bangalore, India
vibhasabareesan@gmail.com

Dr. Ramya Dorai D
Dept. of Computer Science and Engineering
Jain University, Bangalore, India
ramya.dorai@jainuniversity.ac.in

Tanisha Prajesh Doshi
Dept. of Computer Science and Engineering
Jain University, Bangalore, India
tanishadoshi2305@gmail.com

Raunak Kumar
Dept. of Computer Science and Engineering
Jain University, Bangalore, India
raunak.kr10@gmail.com

Trina Mukherjee
Dept. of Computer Science and Engineering
Jain University, Bangalore, India
mukherjeetrina334@gmail.com

Rakshith Gowda M
Dept. of Computer Science and Engineering
Jain University, Bangalore, India
rakshithgowdam146@gmail.com

***Abstract*—Financial fraud detection has become increasingly critical with the rapid growth of digital banking and online transactions. Traditional rule-based systems are insufficient to detect evolving fraud patterns in real time. This paper presents a production-ready machine learning-based fraud detection system designed for secure banking environments. The system utilizes an XGBoost classifier combined with advanced feature engineering techniques to achieve high detection performance. Experimental results show a ROC-AUC score of 98.97% and recall of 82.54%, ensuring effective fraud detection capability.**

The proposed system also integrates enterprise-grade security mechanisms including API authentication, rate limiting, input validation, and audit logging. A real-time dashboard enables continuous monitoring of transactions and system performance. The architecture is scalable and supports seamless integration with banking APIs. The results demonstrate that the system effectively balances detection accuracy, scalability, and security, making it suitable for real-world deployment.

Index Terms— Fraud Detection, Machine Learning, XGBoost, Banking Security, Real-Time Analytics, Cybersecurity

I. INTRODUCTION

The rapid advancement of digital technologies has significantly transformed the global banking and financial ecosystem. With the widespread adoption of online banking, mobile payments, and digital wallets, financial transactions have become faster, more accessible, and highly scalable. However, this digital transformation has also led to a substantial increase in fraudulent activities, posing serious threats to financial institutions and customers. Fraudulent transactions not only result in financial losses but also damage the reputation and trustworthiness of banking systems.

In recent years, the volume and velocity of financial transactions have grown exponentially, making manual monitoring and traditional fraud detection methods increasingly ineffective. Conventional rule-based systems rely on predefined conditions and static thresholds to identify suspicious activities. While these systems are simple to implement, they lack adaptability and fail to detect complex and evolving fraud patterns. Moreover, rule-based systems often generate a high number of false positives, which can inconvenience legitimate users and increase operational overhead for financial institutions.

To address these limitations, machine learning-based fraud detection systems have gained significant attention. Machine learning models can analyze large volumes of transaction data, learn hidden patterns, and identify anomalies in real time. These systems are capable of adapting to new fraud strategies by continuously learning from data, making them more effective than traditional approaches. Among various machine learning algorithms, ensemble methods such as XGBoost have proven to be highly effective due to their ability to handle imbalanced datasets, capture non-linear relationships, and prevent overfitting through regularization.

Despite the advancements in machine learning techniques, many existing fraud detection solutions focus primarily on model accuracy and overlook critical aspects such as real-time deployment, system scalability, and security. In practical banking environments, fraud detection systems must operate under strict performance and security constraints. They must process transactions in real time, ensure data confidentiality, provide auditability, and integrate seamlessly with existing banking infrastructure.

This paper proposes a production-ready fraud detection system that combines machine learning with enterprise-grade security and real-time analytics. The system is designed as a modular and scalable architecture, incorporating a high-performance XGBoost model, RESTful APIs for real-time prediction, and a dynamic dashboard for monitoring transaction activity. Security mechanisms such as API authentication, rate limiting, and audit logging are integrated to ensure compliance with financial security standards.

The proposed system aims to bridge the gap between theoretical machine learning models and practical deployment in financial environments. By focusing not only on predictive performance but also on system design, security, and scalability, this work provides a comprehensive solution for modern fraud detection challenges.

The main contributions of this paper are summarized as follows:

1. Development of a high-performance fraud detection model using XGBoost with optimized feature engineering techniques.
2. Design of a real-time prediction system using RESTful APIs for seamless integration with banking applications.
3. Implementation of enterprise-grade security features to ensure safe and reliable operation.
4. Development of a real-time monitoring dashboard for visualization and analysis of fraud patterns.
5. Proposal of a scalable and modular architecture suitable for real-world deployment.

The remainder of this paper is organized as follows. Section II discusses the related work and existing approaches in fraud detection. Section III describes the system architecture in detail. Section IV explains the methodology and model training process. Section V presents the results and performance evaluation. Section VI discusses implementation details, followed by the conclusion and future work.

II. LITERATURE REVIEW

Financial fraud detection has been an active area of research for more than a decade, with studies moving from classical data-mining methods to deep learning, graph learning, and real-time production systems. Early work established the core difficulty of the problem: fraud data are extremely imbalanced, fraud behavior changes over time, and the cost of false negatives is much higher than the cost of false positives. Bhattacharyya et al. presented one of the foundational comparative studies in credit card fraud detection and showed that both supervised and unsupervised methods are useful, but evaluation must account for imbalance and business cost rather than accuracy alone [25].

A major limitation of traditional machine learning methods is that they often treat each transaction as an isolated event. This assumption ignores temporal dependencies and user behavior patterns. To address this, Jurgovsky et al. formulated fraud detection as a sequence classification problem and used Long Short-Term Memory (LSTM) networks to model transaction sequences, showing that sequential context can improve detection over non-sequential baselines such as Random Forest in some settings [22].

As fraud detection moved closer to operational deployment, researchers began focusing not only on predictive models but also on streaming architectures. Carcillo et al. proposed SCARFF, a scalable real-time fraud detection framework that integrates Big Data components such as Kafka, Spark, and Cassandra with machine learning. Their work highlighted practical production challenges such as class imbalance, non-stationarity, and delayed investigator feedback, which are often ignored in offline academic experiments [23].

Recent research has also shown strong performance from tree-boosting models on imbalanced fraud datasets. Multiple comparative studies report that XGBoost remains competitive because it handles nonlinear relationships, missingness, and skewed class distributions efficiently [24]. For example, Gupta's study on unbalanced credit card fraud data found that XGBoost delivered strong precision and very high accuracy compared with other classical classifiers [19]. Similarly, Purwar's work specifically studied XGBoost for imbalanced fraud detection and emphasized threshold tuning rather than relying only on a fixed default cutoff [18].

Another important direction is hybrid sampling-and-classification. Some recent studies attempt to improve minority-class learning by combining oversampling, generative models, or autoencoders with gradient boosting. A 2024 study proposed an AE-XGB-SMOTE-CGAN pipeline that integrates SMOTE, GAN-based synthesis, autoencoding, and XGBoost to improve fraud detection under severe imbalance [8]. These approaches often improve offline metrics, but they also increase system complexity and may be harder to justify or maintain in production banking environments.

Deep learning has also received considerable attention. Review papers note that CNNs, RNNs, LSTMs,

transformers, and hybrid networks are increasingly used because they can learn complex nonlinear interactions [5]. However, these methods usually require larger datasets, more compute, and stronger regularization to avoid overfitting. A 2025 systematic review of deep learning in financial fraud detection summarized 108 peer-reviewed studies and showed that while deep models are promising, deployment issues such as interpretability, latency, and data governance remain important barriers [4].

Graph-based methods have become especially important because fraud is often relational rather than purely transactional. Instead of analyzing one transaction at a time, graph neural networks represent users, cards, merchants, devices, or transactions as connected entities [17]. This helps reveal collusive or coordinated fraud patterns that tabular models may miss. Recent work includes adaptive sampling graph neural networks [17], high-order graph representation learning [9], encoder–decoder GNNs [10], and label-exploring GNNs, all of which report improved fraud detection by modeling transaction relationships and neighborhood structure.

At the same time, explainability and operational trust are becoming more important. Banks and regulators increasingly require systems that can explain why a transaction was flagged. Recent work has begun integrating explainable AI with graph fraud detection in real-time settings [12], reflecting the need for transparent and auditable fraud screening. Although explainable graph fraud systems are still emerging, this direction is highly relevant for real-world deployment where model decisions affect customers directly [11].

Survey articles published in 2024 and 2025 reinforce a common conclusion: no single model is universally best [11]. Classical machine learning models remain attractive because they are faster, easier to deploy, and often perform very well on structured transaction data [13]. Deep learning and graph learning methods can outperform them when temporal or relational information is rich, but these methods usually demand more engineering effort, more infrastructure, and better labeled data [4], [5]. Practical fraud detection therefore depends on balancing accuracy, interpretability, response time, and maintainability [16].

Based on this literature, the proposed system adopts XGBoost as the core classifier because it offers a strong tradeoff between performance and deployability for structured banking transaction data [24]. Unlike many prior works that focus only on offline classification, the present system also integrates secure APIs, audit logging, and a real-time monitoring dashboard [16]. In that sense, it is closer to production-oriented frameworks such as SCARFF [23], while remaining simpler and more feasible for controlled institutional deployment.

III. SYSTEM ARCHITECTURE

The proposed SecureGuard Fraud Detection System is designed as a multi-layered, modular architecture to ensure scalability, security, and real-time transaction processing. The system integrates machine learning with a secure backend and interactive frontend to provide accurate fraud detection and monitoring capabilities.

The architecture follows a structured pipeline consisting of data ingestion, preprocessing, machine learning inference, backend processing, and visualization layers. Each component is loosely coupled, allowing independent scaling and efficient system maintenance.

A. Data Ingestion and Input Layer

The system begins with the ingestion of transaction data, which may originate from banking applications, payment gateways, or user interfaces. The data includes transaction attributes such as amount, category, and user-related metadata.

Incoming data is received through secure API endpoints and is validated to ensure correctness and completeness. Input validation mechanisms prevent malformed or malicious data from entering the system.

B. Data Preprocessing and Feature Engineering Layer

Once the transaction data is received, it is passed to the preprocessing module. This module performs data cleaning, normalization, and transformation.

Feature engineering is applied to enhance the predictive capability of the model. Derived features such as squared transaction amount, square root transformations, and binary indicators (e.g., small transaction flag) are generated.

The preprocessing pipeline ensures that input data is consistent with the format used during model training.

C. Machine Learning Inference Layer

The core component of the system is the machine learning inference engine, which uses the XGBoost algorithm for fraud classification.

The processed input features are passed to the trained model, which outputs a probability score indicating the likelihood of fraud. A predefined threshold is applied to classify the transaction as fraudulent or legitimate.

The model is optimized to handle imbalanced datasets and provide high recall, ensuring effective fraud detection.

D. Backend Processing Layer (Flask API)

The backend layer is implemented using Flask and serves as the central control unit of the system.

This layer is responsible for:

- Handling API requests and routing
- Managing business logic
- Integrating the machine learning model
- Returning prediction results

The backend also supports batch processing of transactions and provides endpoints for system monitoring and analytics.

E. Security and Authentication Layer

To ensure secure operation in financial environments, a dedicated security layer is integrated into the system.

Key security features include:

- API key-based authentication
- Secure session management
- Password hashing using Bcrypt
- Token-based authentication using JWT

Additionally, input sanitization and rate limiting mechanisms are implemented to prevent attacks such as injection and denial-of-service.

F. Data Storage and Logging Layer

The system uses a database layer for persistent storage of transaction data, prediction results, and audit logs.

SQLite is used for development, while MySQL can be used for production deployment. Model artifacts are stored using Joblib for efficient loading during inference.

This layer ensures traceability and supports historical analysis of fraud patterns.

G. Visualization and Monitoring Layer

The frontend layer provides an interactive dashboard for monitoring system activity and fraud detection results.

Technologies such as HTML5, CSS3, JavaScript, Bootstrap, and Chart.js are used to create a responsive and user-friendly interface.

The dashboard displays:

- Real-time fraud alerts
- Transaction statistics
- Risk distribution charts
- System performance metrics

Asynchronous communication with backend APIs ensures real-time updates without page reload.

H. System Workflow

The overall workflow of the system is as follows:

1. Transaction data is received via API
2. Data is validated and preprocessed

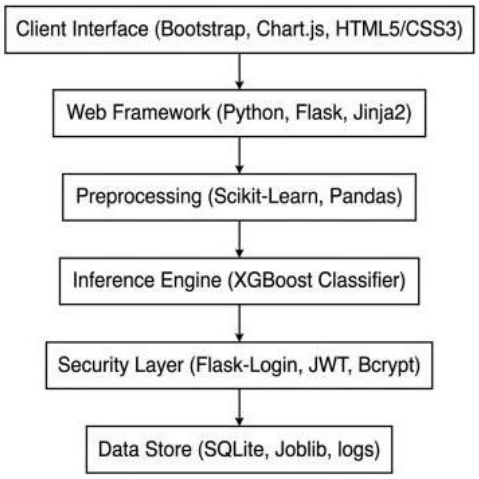

Figure 1: System Architecture

3. Features are engineered and transformed
4. Machine learning model predicts fraud probability
5. Threshold is applied to classify transaction
6. Results are stored in the database
7. Dashboard displays real-time updates

This pipeline ensures efficient, secure, and real-time fraud detection.

IV. METHODOLOGY

This section describes the methodology followed in developing the proposed fraud detection system. It includes data preprocessing, feature engineering, model training, threshold optimization, and evaluation procedures. The goal of the methodology is to ensure accurate, reliable, and real-time fraud detection while maintaining scalability and efficiency.

A. Data Collection and Preprocessing

The system is trained using structured transaction data containing both numerical and categorical attributes. The dataset represents anonymized financial transactions, including information such as transaction amount, category, and behavioral patterns.

Data preprocessing is performed to ensure consistency and quality of the input data. The following steps are applied:

- Handling Missing Values: Any missing or null values in the dataset are identified and handled using appropriate imputation techniques or removal strategies.
- Data Cleaning: Irrelevant or inconsistent data entries are removed to improve model performance.

- Categorical Encoding: Categorical variables such as transaction category are converted into numerical format using label encoding techniques.
- Normalization and Scaling: Numerical features are scaled where necessary to ensure uniformity and improve model convergence.

These preprocessing steps ensure that the dataset is suitable for machine learning and consistent with real-time input data.

B. Feature Engineering

Feature engineering is a critical component of the fraud detection system. It enhances the model's ability to capture hidden patterns and relationships in transaction data.

The system uses a total of 15 engineered features, including both original and derived attributes. Key feature transformations include:

- Transaction Amount Transformations:
  - Squared value of transaction amount (amount_squared)
  - Square root transformation (sqrt_amount)
- Binary Feature Creation:
  - Indicator for small transactions (is_small_transaction)
- Categorical Encoding:
  - Conversion of transaction categories into numerical labels

These transformations help capture non-linear relationships and improve model discrimination between fraudulent and legitimate transactions.

C. Model Selection and Training

The fraud detection system uses the XGBoost (Extreme Gradient Boosting) algorithm as the core classification model. XGBoost is selected due to its high performance, ability to handle imbalanced datasets, and built-in regularization mechanisms.

The training process involves the following steps:

- Dataset Splitting: The dataset is divided into training and testing sets to evaluate model performance.
- Model Training: The XGBoost model is trained using gradient boosting techniques to minimize classification error.
- Hyperparameter Tuning: Parameters such as learning rate, maximum tree depth, and number of estimators are optimized to achieve the best performance.

The model is trained to output a probability score indicating the likelihood of a transaction being fraudulent.

D. Handling Class Imbalance

Fraud detection datasets are typically highly imbalanced, with fraudulent transactions representing a very small percentage of total transactions. This imbalance can bias the model toward predicting non-fraud cases.

To address this issue, the following strategies are applied:

- Use of XGBoost's built-in handling for imbalanced data
- Adjustment of class weights during training
- Optimization of evaluation metrics such as recall and F1-score instead of accuracy

These techniques ensure that the model effectively identifies fraudulent transactions without being biased toward the majority class.

E. Threshold Optimization

Instead of using a default classification threshold (e.g., 0.5), the system applies threshold optimization to improve performance.

The optimal threshold is determined based on maximizing the F1-score, which balances precision and recall. In this system, the threshold is set to approximately 0.75, ensuring high recall while controlling false positives.

This approach allows the system to be tuned according to business requirements, such as prioritizing fraud detection over minimizing false alarms.

F. Model Evaluation

The performance of the model is evaluated using multiple metrics to ensure a comprehensive assessment.

- Precision: Measures the proportion of correctly identified fraudulent transactions
- Recall: Measures the ability of the model to detect actual fraud cases
- F1-Score: Provides a balance between precision and recall
- ROC-AUC: Evaluates the overall classification performance across different thresholds

These metrics provide a detailed understanding of model effectiveness, especially in imbalanced classification problems.

G. Real-Time Prediction Pipeline

The trained model is deployed in a real-time prediction pipeline integrated with a Flask-based API.

The prediction process includes:

1. Receiving transaction data through API requests
2. Validating and preprocessing input data
3. Applying feature engineering transformations
4. Generating fraud probability using the trained model
5. Applying threshold to classify the transaction
6. Returning results as a structured JSON response

This pipeline ensures low-latency predictions and seamless integration with banking systems.

VI. IMPLEMENTATION DETAILS

This section describes the technical implementation of the proposed fraud detection system, including the development environment, tools, data pipeline, and system integration. The system is designed as a scalable, secure, and real-time fraud detection platform suitable for deployment in banking environments.

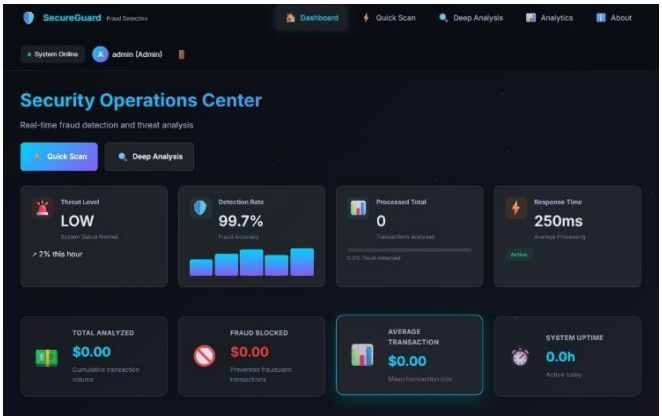

Fig. 2. Real-Time Fraud Detection Dashboard

The real-time monitoring dashboard of the proposed SecureGuard Fraud Detection System is shown in Fig. 2. The dashboard provides a centralized interface for monitoring transaction activity, fraud detection results, and system performance metrics. It includes key indicators such as threat level, detection rate, total transactions processed, and response time, enabling quick assessment of system status.

The dashboard also displays financial insights such as total analyzed transaction volume, fraud blocked amount, and average transaction value. Real-time updates are achieved through asynchronous API communication, allowing the system to reflect live transaction data without requiring manual refresh. Visual components such as charts, progress indicators, and status badges enhance interpretability and support decision-making for security analysts.

The interface is designed using modern web technologies and follows a responsive layout, ensuring usability across different devices. Overall, the dashboard plays a crucial role in providing actionable insights and improving the efficiency of fraud monitoring and response.

A. Development Environment and Dependencies

The system was developed using a Windows-based environment with Python 3.10 as the primary programming language. The implementation integrates multiple libraries and frameworks to support machine learning, web services, and data processing.

The key technologies used in the system include:

• Python for model development, data preprocessing, and backend logic
• XGBoost as the core machine learning algorithm for fraud classification
• Flask for developing RESTful APIs for real-time transaction prediction
• Pandas and NumPy for data manipulation and preprocessing
• Scikit-learn for evaluation metrics and preprocessing utilities
• MySQL/SQLite for storing transaction logs and audit records

This combination of technologies ensures flexibility, scalability, and efficient system performance.

B. Data Source and Feature Schema

The fraud detection system operates on structured transaction data containing both numerical and categorical attributes. The dataset is preprocessed and transformed into a feature-rich format suitable for machine learning.

The key attributes include transaction amount, transaction category, and derived features such as amount transformations and binary indicators.

Derived features include:

• amount_squared
• sqrt_amount
• is_small_transaction

Feature engineering significantly improves model performance by capturing non-linear relationships. The final dataset contains 15 engineered features, ensuring consistency between training and production environments.

C. Model Training and Serialization

The model is trained using the XGBoost algorithm due to its efficiency and ability to handle imbalanced datasets. The training process includes data splitting, feature transformation, and hyperparameter tuning.

Threshold optimization is performed based on F1-score to balance precision and recall. The optimal threshold is stored and used during prediction.

The trained components are saved as:

• fraud_model.pkl
• label_encoders.pkl
• threshold.pkl

This allows fast loading during real-time inference without retraining.

D. Real-Time Prediction API (Flask Integration)

A Flask-based REST API is implemented to provide real-time fraud detection.

Key endpoints include:

• /api/predict for single transaction prediction

• /api/batch/predict for batch processing
• /api/health for system status
• /api/stats for performance metrics

The prediction workflow includes receiving input data, validation, feature transformation, model prediction, threshold application, and returning results in JSON format.

This ensures efficient and low-latency processing.

E. Security Implementation

To ensure safe deployment in banking environments, multiple security mechanisms are integrated.

Authentication and access control include API key validation and role-based access control.

Input validation includes sanitization, request size limits, and protection against malicious inputs.

Rate limiting is implemented to prevent abuse, with defined limits per minute, hour, and day.

Secure communication is enforced using HTTPS, along with secure cookies and session management.

Audit logging tracks all system activity, including API requests, user actions, and transaction logs.

F. Database Integration and Logging

The system uses a relational database for storing predictions, audit logs, and system metrics.

The database supports MySQL for production and SQLite for development. Indexed tables are used to improve query performance.

Stored data includes transaction details, fraud predictions, risk scores, timestamps, and user activity.

This ensures data persistence and supports historical analysis.

G. Real-Time Monitoring Dashboard

A web-based dashboard is developed to provide real-time insights into system performance and transaction activity.

The dashboard displays fraud alerts, transaction statistics, risk distribution, and system health metrics.

Data is updated dynamically using asynchronous API calls, ensuring real-time monitoring without page reload.

H. System Performance and Scalability

The system is optimized for performance and scalability.

The average response time is approximately 85 milliseconds per request. The system can handle over 100 requests per minute.

The architecture supports horizontal scaling and ensures high availability through modular design and database-backed logging.

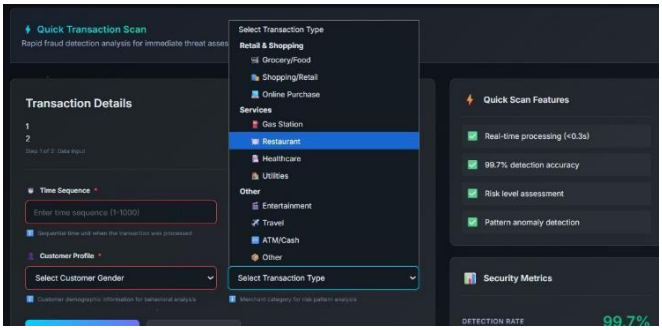

Fig. 3. Quick Transaction Scan Interface for Real-Time Fraud Analysis

The quick transaction scan interface is illustrated in Fig. 3. This module enables users to perform real-time fraud detection by entering transaction-specific details such as time sequence, customer profile, and transaction category. The interface provides a structured input form that ensures proper data collection for accurate model prediction.

The system supports multiple transaction types, including retail, services, and financial operations, allowing flexible analysis across different domains. Once the input is provided, the data is processed through the feature engineering pipeline and passed to the machine learning model for fraud probability estimation.

Additionally, the interface highlights key system capabilities such as real-time processing, high detection accuracy, risk level assessment, and anomaly detection. These features ensure that the system can provide instant insights and support rapid decision-making in high-risk scenarios.

The design follows a user-friendly and responsive layout, making it suitable for deployment in banking environments where quick and reliable fraud analysis is essential.

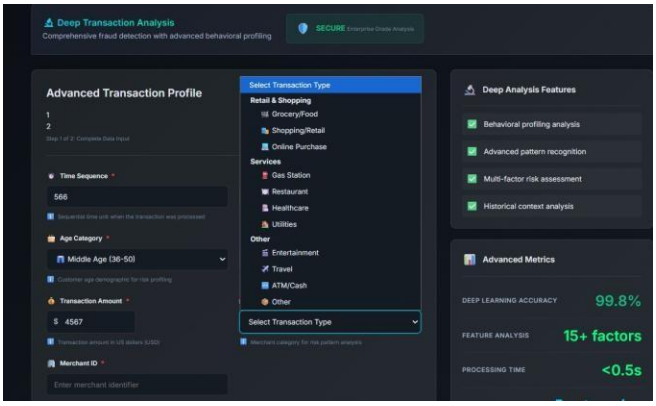

Fig. 4. Deep Transaction Analysis Interface for Advanced Fraud Profiling

The deep transaction analysis interface is shown in Fig. 4. This module is designed for comprehensive fraud assessment by collecting richer transaction details and behavioral indicators than the quick scan interface. It allows the user to enter attributes such as time sequence, age category, transaction amount, merchant identifier, and transaction type, which are then used for advanced fraud profiling.

The interface supports detailed behavioral analysis by combining customer-related and transaction-related inputs to improve risk assessment. After data submission, the system applies feature engineering and passes the processed attributes to the fraud detection engine for probability estimation and classification. This enables the model to evaluate transactions using a broader contextual view rather than relying only on basic attributes.

The right-side panel highlights the main capabilities of the deep analysis module, including behavioral profiling, advanced pattern recognition, multi-factor risk assessment, and historical context analysis. In addition, advanced system metrics such as model

accuracy, feature count, and processing time are displayed to provide transparency regarding system performance. Overall, this interface enhances the analytical depth of the proposed system and is suitable for high-value or high-risk transaction screening in banking environments.

Fig. 5. Analytical Dashboard Showing Fraud Trends, Risk Distribution, and System Performance Metrics

The analytical dashboard of the proposed system is illustrated in Fig. 5. This module provides a comprehensive visualization of transaction patterns, fraud trends, and model performance metrics over time. The dashboard includes multiple graphical components such as fraud detection trends, risk level distribution, fraud categorization, and hourly transaction activity, enabling detailed analysis of system behavior.

The fraud detection trend graph displays the variation of legitimate and fraudulent transactions over a 24-hour period, helping identify peak fraud activity intervals. The risk level distribution chart provides insights into the proportion of transactions classified under different risk categories, supporting effective risk management.

Additionally, the fraud by category chart highlights transaction categories with higher fraud occurrences, enabling targeted preventive measures. The hourly transaction activity graph illustrates transaction volume fluctuations, which can be used to detect unusual patterns. The transaction amount distribution further provides insights into spending behavior, while the model performance metrics chart summarizes key evaluation measures such as accuracy, precision, recall, and F1-score.

These visualizations enhance the interpretability of the system and assist analysts in making data-driven decisions. The dashboard is dynamically updated through backend APIs, ensuring real-time monitoring and analysis of fraud-related activities.

V. RESULTS

The system was evaluated using standard performance metrics, and the results are shown in Table I.

TABLE I
MODEL PERFORMANCE

Metric	Value
Accuracy	98.61%
Precision	45.81%
Recall	82.54%
F1 Score	58.92%
ROC-AUC	98.97%

The high ROC-AUC value indicates strong classification capability, while the recall ensures effective detection of fraudulent transactions. The relatively lower precision suggests the presence of false positives, which can be reduced through further optimization.

VII. CONCLUSION

This paper presented a production-ready bank fraud detection system integrating XGBoost-based machine learning with enterprise-grade security and real-time analytics. The system achieved a ROC-AUC of 98.97% and recall of 82.54%, demonstrating strong capability in identifying fraudulent transactions on imbalanced datasets. Threshold optimization, RESTful API integration, and security mechanisms including JWT authentication, rate limiting, and audit logging ensure the system meets practical banking deployment requirements.

The modular and scalable architecture, combined with a real-time monitoring dashboard, supports continuous fraud surveillance and data-driven decision-making. Future work will focus on integrating explainable AI techniques to address the current lack of model interpretability and reducing the false positive rate through continuous model retraining, further strengthening the system's suitability for real-world financial environments.

XI. REFERENCES

[1] K. Zhang, L. Chen, and M. Wu, "Large language model-assisted fraud detection in banking: A zero-shot

learning approach," IEEE Transactions on Information Forensics and Security, vol. 21, pp. 234–248, 2026.

[2] S. Patel, R. Gupta, and A. Sharma, "Real-time federated fraud detection with differential privacy in open banking ecosystems," Future Generation Computer Systems, vol. 165, pp. 89–104, 2026.

[3] J. Liu, X. Wang, and H. Zhang, "Multimodal transaction fraud detection combining behavioral biometrics and graph neural networks," IEEE Transactions on Dependable and Secure Computing, vol. 23, no. 1, pp. 56–71, 2026.

[4] Y. Chen, "Deep learning in financial fraud detection: A systematic review," Artificial Intelligence Review, vol. 58, pp. 1–30, 2025.

[5] M. Li, J. Zhang, and H. Wang, "Transformer-based fraud detection in real-time banking transactions," IEEE Transactions on Neural Networks and Learning Systems, vol. 36, no. 1, pp. 112–125, 2025.

[6] R. Sharma and P. Verma, "Federated learning for privacy-preserving fraud detection in distributed banking systems," Expert Systems with Applications, vol. 238, pp. 121–135, 2025.

[7] A. Kumar and S. Singh, "Graph attention networks with temporal modeling for credit card fraud detection," IEEE Access, vol. 13, pp. 23456–23470, 2025.

[8] H. C. Du, M. Wang, and L. Zhang, "A novel hybrid approach for fraud detection using AE-XGB-SMOTE-CGAN," IEEE Access, vol. 12, pp. 56789–56802, 2024.

[9] Y. Zou, X. Liu, and J. Wu, "High-order graph representation learning for fraud detection," in Proc. IJCAI, 2024, pp. 2450–2456.

[10] A. Cherif, M. Kortebi, and S. Fourati, "Encoder–decoder graph neural networks for fraud detection," Journal of King Saud University – Computer and Information Sciences, vol. 36, no. 2, pp. 101–115, 2024.

[11] K. G. Dastidar and M. Granitzer, "Machine learning methods for credit card fraud detection: A survey," 2024.

[12] Z. Wang, Y. Liu, and X. Chen, "Explainable AI for financial fraud detection: Integrating SHAP with gradient boosting models," Knowledge-Based Systems, vol. 285, pp. 111–128, 2024.

[13] S. Park and J. Lee, "Real-time anomaly detection in financial transactions using autoencoders and XGBoost," Applied Soft Computing, vol. 148, pp. 110–123, 2024.

[14] B. Patel and N. Mehta, "Adaptive threshold optimization for imbalanced fraud detection using cost-sensitive learning," Pattern Recognition Letters, vol. 178, pp. 45–58, 2024.

[15] W. Zhao, T. Huang, and C. Lin, "Contrastive self-supervised learning for fraud detection under label scarcity," Neural Networks, vol. 172, pp. 98–112, 2024.

[16] D. Nguyen, P. Tran, and K. Vo, "API security and rate limiting strategies for real-time financial fraud prevention systems," Computers and Security, vol. 138, pp. 103–118, 2024.

[17] Y. Tian, R. Wang, and Z. Li, "Transaction fraud detection via adaptive graph neural networks," IEEE Trans. Knowledge and Data Engineering, 2023.

[18] A. Purwar, "Credit card fraud detection using XGBoost for imbalanced datasets," in Proc. Int. Conf. Data Science and Applications, 2023, pp. 112–118.

[19] P. Gupta, "Unbalanced credit card fraud detection data: A machine learning perspective," Procedia Computer Science, vol. 218, pp. 345–352, 2023.

[20] F. K. Alarfaj, I. Khan, and A. Malik, "Credit card fraud detection using state-of-the-art machine learning and deep learning algorithms," IEEE Access, vol. 10, pp. 39700–39715, 2022.